

Lista de Exercícios C2-FINAL — SOLID em Python

Parte 1 — SRP e OCP

Exercício 1 — Sistema de Pedidos

Uma loja possui o seguinte código:

Python

```
class Pedido:
    def __init__(self, cliente, valor):
        self.cliente = cliente
        self.valor = valor
```

Python

```
def calcular_total(self):
    return self.valor * 1.10
```

Python

```
def salvar(self):
    print("Salvando pedido no banco de dados...")
```

Python

```
def enviar_email(self):
    print(f"Enviando confirmação para {self.cliente}...")
```

O sistema funciona, porém a classe `Pedido` possui várias responsabilidades.

Tarefa:

1. Identifique as responsabilidades existentes na classe `Pedido`.
2. Explique por que ela viola o **SRP**.
3. Refatore o código separando as responsabilidades.
4. Crie pelo menos as classes:
 - `Pedido`
 - `PedidoRepository`
 - `EmailService`
5. Crie um programa principal que demonstre o funcionamento das classes.

Conceito principal: S – Single Responsibility Principle.

Exercício 2 — Calculadora de Frete

Analise:

Python

```
class CalculadoraFrete:
```

Python

```
def calcular(self, tipo, valor):  
    if tipo == "normal":  
        return valor * 0.05  
  
    elif tipo == "expresso":  
        return valor * 0.10  
  
    elif tipo == "sedex":  
        return valor * 0.15  
  
    return 0
```

Toda vez que uma nova modalidade de frete é criada, é necessário alterar `CalculadoraFrete`.

Tarefa:

Refatore o sistema utilizando **OCP**.

Crie uma estrutura que permita utilizar:

```
Python
FreteNormal
FreteExpresso
FreteSedex
```

Depois, faça com que o seguinte código funcione:

```
Python
frete = FreteExpresso()

valor_frete = calcular_frete(frete, 1000)

print(valor_frete)
```

Finalmente, crie uma nova modalidade:

```
Python
FreteInternacional
```

A nova modalidade deve cobrar **20%** do valor.

Ela deve ser adicionada **sem modificar as classes de frete que já existem**.

Conceito principal: 0 – Open/Closed Principle.

Exercício 3 — Sistema de Funcionários

Uma empresa possui diferentes tipos de funcionários:

```
Python
Desenvolvedor
Gerente
Estagiario
```

Inicialmente foi criada a seguinte classe:

Python

```
class Funcionario:
    def __init__(self, nome, salario, cargo):
        self.nome = nome
        self.salario = salario
        self.cargo = cargo
```

Python

```
def calcular_bonus(self):
    if self.cargo == "desenvolvedor":
        return self.salario * 0.10

    elif self.cargo == "gerente":
        return self.salario * 0.20

    elif self.cargo == "estagiario":
        return self.salario * 0.05
```

Além disso, o sistema precisa:

- salvar funcionários;
- calcular bônus;
- gerar relatório.

Tarefa:

Refatore o sistema aplicando **SRP e OCP**.

O sistema deve permitir:

Python

```
funcionarios = [
    Desenvolvedor("Carlos", 5000),
    Gerente("Maria", 8000),
    Estagiario("João", 2000)
]

for funcionario in funcionarios:
    print(funcionario.calcular_bonus())
```

Depois adicione:

```
Python
Analista
```

O analista recebe **15% de bônus**.

A inclusão do **Analista** não deve exigir alterações nas classes existentes responsáveis pelo cálculo.

Conceitos: **S + 0.**

Exercício 4 — Sistema de Notificações

Um sistema possui a seguinte implementação:

```
Python
class Notificacao:
```

```
Python
    def enviar(self, tipo, mensagem):

        if tipo == "email":
            print(f"Enviando EMAIL: {mensagem}")

        elif tipo == "sms":
            print(f"Enviando SMS: {mensagem}")

        elif tipo == "whatsapp":
            print(f"Enviando WhatsApp: {mensagem}")
```

O sistema está crescendo e futuramente poderá possuir muitas outras formas de notificação.

Tarefa:

1. Identifique o problema relacionado ao **OCP**.
2. Crie classes separadas para:
 - Email
 - SMS
 - WhatsApp
3. Todas devem possuir um método:
- 4.

```
Python
enviar(mensagem)
```

5. Crie uma função:

```
Python
def enviar_notificacao(notificador, mensagem):
    # implementar
```

O programa deve funcionar assim:

```
Python
email = Email()
sms = SMS()

enviar_notificacao(email, "Pedido realizado!")
enviar_notificacao(sms, "Pagamento aprovado!")
```

Depois implemente:

```
Python
PushNotification
```

sem modificar Email, SMS ou WhatsApp.

Conceito principal: O – Open/Closed Principle.

Exercício 5 — Sistema de Pagamentos

Uma loja virtual precisa processar pagamentos.

Atualmente existe:

Python

```
class Pagamento:
```

Python

```
def processar(self, tipo, valor):  
  
    if tipo == "pix":  
        print(f"PIX de R$ {valor}")  
  
    elif tipo == "cartao":  
        print(f"Cartão de R$ {valor}")  
  
    elif tipo == "boleto":  
        print(f"Boleto de R$ {valor}")  
  
    print("Salvando pagamento no banco...")  
  
    print("Enviando comprovante por e-mail...")
```

Esse código apresenta problemas relacionados a **SRP e OCP**.

Tarefa:

Refatore o sistema.

Separe as responsabilidades de:

Python

Pagamento

Persistência

Envio de comprovante

Crie diferentes formas de pagamento:

```
Python
Pix
Cartao
Boleto
```

Cada forma de pagamento deve possuir:

```
Python
processar(valor)
```

Depois adicione:

```
Python
Criptomoeda
```

A nova forma de pagamento deve ser adicionada sem alterar as implementações existentes.

No programa principal, utilize uma lista:

```
Python
pagamentos = [
    Pix(),
    Cartao(),
    Boleto()
]

for pagamento in pagamentos:
    pagamento.processar(500)
```

Conceitos: S + 0.

Parte 2 — LSP

Agora entram os exercícios de **L — Liskov Substitution Principle**.

A ideia central que os alunos devem observar é:

Se uma classe filha herda de uma classe pai, deve ser possível utilizar a filha no lugar da pai sem quebrar o comportamento esperado do programa.

Exercício 6 — Sistema de Aves

Analise:

Python

```
class Ave:

    def voar(self):
        print("A ave está voando.")

class Aguia(Ave):

    def voar(self):
        print("A águia está voando.")

class Pinguim(Ave):

    def voar(self):
        raise Exception("Pinguins não podem voar!")
```

Agora considere:

Python

```
aves = [
    Aguia(),
    Pinguim()
]

for ave in aves:
```

```
ave.voar()
```

O código apresenta um problema relacionado ao **LSP**.

Tarefa:

1. Explique por que **Pinguim** viola o princípio de substituição de Liskov.
2. Refatore a hierarquia.
3. Separe corretamente aves que voam de aves que não voam.
4. O sistema deve continuar permitindo representar:
 - Águia
 - Pardal
 - Pinguim
5. **Pinguim** não deve possuir um método **voar()** que lance exceção apenas porque herdou esse comportamento.

Conceito principal: L – Liskov Substitution Principle.

Exercício 7 — Sistema de Contas Bancárias

Um banco possui:

Python

```
class Conta:

    def __init__(self, saldo):
        self.saldo = saldo

    def depositar(self, valor):
        self.saldo += valor

    def sacar(self, valor):
        self.saldo -= valor

class ContaCorrente(Conta):
    pass
```

```
class ContaInvestimento(Conta):  
  
    def sacar(self, valor):  
        raise Exception("Não é permitido sacar diretamente desta  
conta.")
```

Existe também:

```
Python  
def realizar_saque(conta, valor):  
    conta.sacar(valor)  
    print("Saque realizado!")
```

O seguinte código funciona:

```
Python  
conta = ContaCorrente(1000)  
  
realizar_saque(conta, 200)
```

Mas:

```
Python  
conta = ContaInvestimento(1000)  
  
realizar_saque(conta, 200)
```

quebra o programa.

Tarefa:

1. Explique por que existe uma violação do **LSP**.
2. Analise se `ContaInvestimento` realmente deveria herdar de uma classe que promete possuir `sacar()`.
3. Refatore a hierarquia.
4. Crie uma estrutura que diferencie:
 - contas que permitem depósito;
 - contas que permitem saque;
 - contas de investimento.
5. Demonstre no programa principal que as classes podem ser utilizadas sem depender de exceções para corrigir uma herança inadequada.

Desafio: tente resolver o problema utilizando **herança e abstração**, sem verificar o tipo do objeto com `if`, `type()` ou `isinstance()`.

Conceito principal: L – Liskov Substitution Principle.